Software Engineering Lab

Lab 19

Database Connection Pooling

Overhead Handshake, Batas Server, dan Bahaya Kebocoran Koneksi di Go



Lukman / lukman-ss

Error Koneksi Meledak Saat Metrik CPU Database Normal

Aplikasi menolak jutaan pengguna meski utilitas server database masih sangat rendah.

PARADOKS KEGAGALAN KONEKSI BASIS DATA

Penolakan Slot Keras

PostgreSQL melempar galat FATAL: too many clients saat slot max_connections terisi penuh.

PARADOKS KEGAGALAN KONEKSI BASIS DATA

Bukan Berarti Rusak

Galat koneksi bukan berarti server kehabisan komputasi CPU, melainkan alokasi slot backend habis.

PARADOKS KEGAGALAN KONEKSI BASIS DATA

Refleks Salah Tim

Menaikkan ukuran pool secara membabi buta justru memperparah kontensi lock dan memicu degradasi.

Banyak insiden produksi ditandai dengan ledakan galat koneksi sementara CPU database berada di bawah 30 persen. Fenomena ini bukan berarti server mengalami kerusakan komputasi internal, melainkan slot koneksi habis dialokasikan oleh kumpulan worker aplikasi yang tidak terkoordinasi.

Connection limits represent hard operating system socket and memory slots rather than raw processing power.

Pendekatan Naif Membuka Koneksi Per-Request Menghancurkan Latensi

Membuat koneksi baru pada setiap query menumpuk beban TCP dan TLS handshake.

TITIK RUNTUH KONEKSI TANPA POOLING

Overhead Berulang

Setiap koneksi menuntut negosiasi TCP 3-way handshake, TLS exchange, serta autentikasi peran.

TITIK RUNTUH KONEKSI TANPA POOLING

Tekanan Backend

Database wajib membuat proses backend baru dan mengalokasikan work_mem untuk tiap request.

TITIK RUNTUH KONEKSI TANPA POOLING

Tidak Otomatis Cepat

Membuka koneksi baru tidak otomatis menyelesaikan beban; antrean pembuatan soket memperlambat IO.

Pendekatan naif tanpa reuse mengabaikan besarnya biaya inisialisasi jaringan dan alokasi memori proses basis data. Membuka koneksi baru untuk setiap request tidak otomatis mempercepat eksekusi, melainkan mengubur runtime aplikasi dalam tumpukan latency handshake berulang.

Direct connection creation incurs recurring network handshakes and expensive OS process allocation penalties.

Model Mental Restoran: Sizing Ditentukan Hardware Backend

Kapasitas koneksi dibatasi oleh inti CPU dan disk, bukan nafsu request klien.

HARDWARE CPU

1. Inti Komputasi

Koneksi konkuren efektif dibatasi core fisik; rumus ((core * 2) + spindle) jadi acuan awal.

SQL.DB SEMAPHORE

2. Client Pool

Mengatur antrean peminjam koneksi lokal; mencegah goroutine menyerbu database serentak.

DEGRADASI KURVA

3. Saturation Knee

Melewati titik jenuh, throughput anjlok drastis akibat context switching dan lock contention.

UKURAN MINIMAL

4. Steady State

Hanya pada jalur ini pool berukuran ringkas menghasilkan throughput optimal dan stabil.

Koneksi database bekerja layaknya meja restoran di mana kapasitas dibatasi oleh jumlah juru masak dapur, bukan antrean tamu di depan pintu. Hanya pada jalur ini penggunaan pool berukuran ringkas mampu membatasi perebutan sumber daya internal dan menghasilkan kestabilan throughput.

Size connection pools to match backend hardware concurrency constraints rather than client request volume.

Siklus Hidup Peminjaman: Pinjam, Eksekusi, dan Kembalikan

Arsitektur pool mengelola antrean akuisisi dan pemakaian ulang koneksi aktif.

ACQUIRE TOKEN

1. Request Conn

Worker meminta koneksi aktif via pool.Conn(ctx); mengambil idle atau membuat baru.

EXECUTE WORK

2. Bounded Query

Query dieksekusi secara singkat tanpa operasi I/O eksternal di dalam siklus penguncian.

DEFER CONN.CLOSE()

3. Immediate Return

Koneksi dikembalikan ke daftar idle pool; siap dipakai ulang oleh worker berikutnya.

PGBOUNCER

4. Proxy Layer

Proxy menerjemahkan ribuan pool aplikasi; lab belum membuktikan multi-node proxy riil.

Setiap transaksi wajib meminjam koneksi hanya untuk eksekusi query spesifik lalu segera mengembalikannya ke pool via defer conn.Close(). Simulasi lab memvalidasi siklus peminjaman lokal ini, namun belum membuktikan routing transaksi dinamis pada kluster PgBouncer multi-node di produksi.

Connections must be borrowed for minimal durations and returned immediately to maintain pool liquidity.

Pola Eksekusi Bersih: Isolasi I/O Eksternal dari Koneksi

Memisahkan panggilan jaringan pihak ketiga dari siklus penahanan koneksi database.

```
func (s *OrderService) ProcessOrderSafe(ctx context.Context, orderID int, externalCall func() error) error {
    if externalCall != nil {
        if err := externalCall(); err != nil { return err }
    }
    conn, err := s.db.Conn(ctx)
    if err != nil { return err }
    defer conn.Close()
    _, err = conn.ExecContext(ctx, "UPDATE orders SET status = 'completed' WHERE id = ?", orderID)
    return err
}
```

Fungsi ProcessOrderSafe menuntaskan panggilan API eksternal terlebih dahulu sebelum meminjam koneksi dari database pool. Menunda akuisisi koneksi bukan berarti menambah latensi alur, melainkan menjamin koneksi database hanya ditahan selama beberapa mikrodetik pada saat query dieksekusi.

Acquire database connections at the absolute latest moment and release them at the earliest opportunity.

internal/pool/service.go:16–34

Kronologi Kebocoran Koneksi yang Memicu Starvation

Satu fungsi tidak disiplin memblokir seluruh request sehat hingga batas timeout.

ALUR KEGAGALAN AKIBAT KONEKSI TERTOMBAK

Langkah 1: Tahan Soket

Worker A meminjam koneksi, menjalankan UPDATE, lalu memanggil API lambat tanpa melepas soket.

ALUR KEGAGALAN AKIBAT KONEKSI TERTOMBAK

Langkah 2: Pool Kritis

Semua slot pool terisi oleh worker yang menanti respons jaringan pihak ketiga (500ms).

ALUR KEGAGALAN AKIBAT KONEKSI TERTOMBAK

Langkah 3: Antrean Tercekat

Worker B yang memiliki kode benar mencoba meminjam koneksi namun masuk antrean tunggu.

ALUR KEGAGALAN AKIBAT KONEKSI TERTOMBAK

Langkah 4: Context Timeout

Hanya pada jalur ini Worker B gugur dengan galat Deadline Exceeded akibat pool starvation.

Menahan koneksi database selama pemrosesan I/O pihak ketiga adalah racun arsitektur yang melumpuhkan ketersediaan pool. Hanya pada jalur ini request sehat yang kodenya bersih ikut mati akibat context deadline exceeded karena kehabisan slot koneksi yang tersandera.

Holding shared connection slots during slow external I/O creates systemic pool starvation for healthy requests.

Matriks Diagnostik: Patologi Koneksi Basis Data

Membedakan karakteristik teknis antara unpooled, oversized pool, dan connection leak.

Dimensi Masalah	Unpooled (No Reuse)	Oversized Pool	Connection Leak (Starvation)
Mekanisme Akar	SetMaxIdleConns(0) tutup koneksi	Client pool > server limit	Koneksi ditahan saat I/O eksternal
Gejala di Server	CPU spike akibat handshake TCP	Error: too many clients	CPU database terlihat normal/idle
Dampak Latensi	Latensi baseline tinggi di p99	Antrean lock & context switch	Lonjakan waktu tunggu akuisisi pool
Manifestasi Error	Koneksi gagal di lonjakan tinggi	ErrServerOverloaded / PG 53300	Context Deadline Exceeded pada klien
Mitigasi Utama	Aktifkan idle pool reuse	Hitung sizing total deployment	Isolasi I/O & leakDetectionThreshold

Tabel ini memetakan perbedaan akar masalah antara tanpa pool, pool terlalu besar, dan kebocoran penahanan koneksi. Kebocoran koneksi memiliki karakteristik paling berbahaya karena server database terlihat sehat padahal antrean klien mengalami kelaparan ekstrem.

Diagnose connection anomalies by correlating application pool wait times against database backend slot usage.

Tiga Prinsip Esensial Penentuan Ukuran Pool

Hierarki keputusan rekayasa untuk mengamankan stabilitas koneksi sistem.

PANDUAN PENETAPAN SIZING KONEKSI

1. Hitung Skala Global

Total koneksi = (jumlah instance × pool size); wajib di bawah max_connections database.

PANDUAN PENETAPAN SIZING KONEKSI

2. Sisihkan Slot Cadangan

Kurangi minimal 15 koneksi untuk superuser, replikasi fisik, dan monitoring sistem.

PANDUAN PENETAPAN SIZING KONEKSI

3. Formula Titik Awal

Gunakan ((core * 2) + disk); bukan berarti angka mutlak, melainkan basis uji beban.

Penetapan ukuran pool wajib memperhitungkan total armada aplikasi yang terpasang pada kluster produksi. Mengadopsi rumus ((core * 2) + disk) bukan berarti mendapatkan angka sakral tanpa cela, melainkan menetapkan baseline terukur yang wajib divalidasi lewat pengujian beban.

Enforce deployment-wide connection limits and validate core-based sizing formulas through empirical stress tests.

Verifikasi Eksekusi: Test Suite dan CLI Demo Lolos Bersih

Output terminal membuktikan penalti unpooled, penolakan server, dan deteksi starvation.

```
=== RUN   TestDirectConnectionOverhead
--- PASS: TestDirectConnectionOverhead (0.03s) [pooled faster than unpooled]
=== RUN   TestOversizedPoolExhaustsServerConnections
--- PASS: TestOversizedPoolExhaustsServerConnections (0.02s) [10 rejected]
=== RUN   TestConnectionStarvationDueToLeak
--- PASS: TestConnectionStarvationDueToLeak (0.02s) [deadline exceeded]
=== RUN   TestTotalCreatedPoolReuse
--- PASS: TestTotalCreatedPoolReuse (0.00s) [reuse verified: 1 vs 5]
PASS: ok  labs/19-database-connection-pooling/tests  1.328s (race clean)
1. Unpooled: 54.59ms vs Pooled: 6.33µs
2. Oversized Pool: Attempted 30, Succeeded 15, Server Rejected 15
3. Starvation: Order 3 Failed: context deadline exceeded
```

Hasil eksekusi membuktikan sepuluh skenario uji lolos sempurna tanpa data race di bawah Go race detector. Demonstrasi CLI mengonfirmasi keunggulan reuse mikrodetik atas handshake milidetik, penolakan tepat 15 request saat server saturasi, serta isolasi kegagalan pada kebocoran koneksi.

Deterministic mock testing provides empirical verification of connection lifecycle bottlenecks and exhaustion limits.

Batasan Bukti Lab: Yang Dibuktikan vs Belum

Menegaskan batas simulasi in-memory terhadap kompleksitas database PostgreSQL riil.

BATAS PEMBUKTIAN SISTEM LAB 19

YANG DIBUKTIKAN

Overhead koneksi baru, penolakan slot server, pool starvation, dan penanganan double close.

BATAS PEMBUKTIAN SISTEM LAB 19

YANG TIDAK DIBUKTIKAN

Alokasi work_mem per backend, lock contention engine, OOM PostgreSQL, dan TCP keepalive.

BATAS PEMBUKTIAN SISTEM LAB 19

BATAS KLAIM EMPIRIS

Lab belum membuktikan efektivitas formula core pada NVMe SSD modern berkecepatan tinggi.

Lab ini membuktikan mekanika dasar peminjaman pool dan batas penolakan slot menggunakan mock driver in-memory. Pengujian ini belum membuktikan perilaku internal mesin database PostgreSQL nyata seperti pemborosan memori work_mem atau saturasi IOPS pada SSD NVMe.

Distinguish application connection pool queuing from internal database engine process and memory management.

Kompromi Rekayasa: Pool Kecil vs Pool Besar

Mengevaluasi konsekuensi operasional antara antrean antarmuka dan kestabilan backend.

ANALISIS PERBANDINGAN TRADE-OFF

Pool Kecil: Protektif

Database beroperasi prima tanpa kejenuhan; request yang meluap wajib sabar mengantre di aplikasi.

ANALISIS PERBANDINGAN TRADE-OFF

Pool Besar: Ilusi Konkurensi

Menampung lonjakan sesaat, namun memicu kontensi lock parah dan risiko kehabisan RAM.

ANALISIS PERBANDINGAN TRADE-OFF

Tidak Otomatis Aman

Memperbesar pool tidak otomatis menaikkan throughput bila kapasitas eksekusi disk database jenuh.

Memilih ukuran pool adalah seni menyeimbangkan antrean di sisi aplikasi melawan stabilitas mesin basis data. Memperbesar ukuran pool tidak otomatis meningkatkan kecepatan pemrosesan jika konkurensi tambahan tersebut hanya memicu perkelahian lock internal dan thrashing memori.

Trade client-side acquisition latency for backend stability by maintaining compact connection pools.

Mekanisme Pengendali Koneksi Menggunakan Atomic Counter

Simulasi batas server dan pencatatan koneksi aktif dengan perlindungan thread-safe.

```
func (d *MockDriver) Open(name string) (driver.Conn, error) {
    if d.connectDelay > 0 { time.Sleep(d.connectDelay) }
    d.mu.Lock()
    defer d.mu.Unlock()
    if d.maxConnections > 0 && atomic.LoadInt32(&d.activeConns) >= d.maxConnections {
        return nil, ErrServerOverloaded
    }
    atomic.AddInt32(&d.activeConns, 1)
    atomic.AddInt32(&d.totalCreated, 1)
    return &mockConn{driver: d}, nil
}
```

MockDriver mengendalikan pembukaan koneksi dengan menyimulasikan delay handshake dan menolak permintaan saat batas activeConns tercapai. Simulasi ini tidak sama dengan proses forking postgres di Linux, namun secara akurat merefleksikan penolakan slot keras pada beban tinggi.

Atomic tracking of active socket handles enforces deterministic connection bounds across concurrent goroutines.

internal/pool/mockdb.go:33–49

Transformasi: Dari Koneksi Bocor ke Eksekusi Terisolasi

Pergeseran dari penahanan soket sembrono menuju disiplin peminjaman minimal.

KOMPARASI KONDISI EKSEKUSI LAYANAN

Sebelum: Tahan Sembrono

Koneksi dipinjam sebelum API pihak ketiga dipanggil; soket tersandera selama ratusan milidetik.

KOMPARASI KONDISI EKSEKUSI LAYANAN

Sesudah: Peminjaman Sempit

API eksternal diselesaikan lebih dulu; koneksi hanya dipinjam selama mikrosekon eksekusi SQL.

KOMPARASI KONDISI EKSEKUSI LAYANAN

Bukan Berarti Lambat

Memecah alur bukan berarti memperlambat kode; pool terbebas dari ancaman kelaparan sistemik.

Sebelum refaktorisasi, koneksi database disandera selama panggilan jaringan eksternal berlangsung lambat. Memecah alur eksekusi bukan berarti menambah kompleksitas, melainkan membebaskan pool dari ancaman starvation sehingga puluhan transaksi lain dapat berjalan mulus.

Isolate slow I/O boundaries outside database connection scopes to preserve shared pool availability.

Anatomi Kode Berbahaya: Menyandera Koneksi Saat I/O Lambat

Pola anti-pattern ProcessOrderUnsafeLeak yang mengunci soket database.

```
func (s *OrderService) ProcessOrderUnsafeLeak(ctx context.Context, orderID int, externalCall func() error) error {
    conn, err := s.db.Conn(ctx)
    if err != nil { return err }
    defer conn.Close()
    _, err = conn.ExecContext(ctx, "UPDATE orders SET status = 'processing' WHERE id = ?", orderID)
    if err != nil { return err }
    if externalCall != nil {
        if err := externalCall(); err != nil { return err }
    }
    return nil
}
```

Metode ProcessOrderUnsafeLeak menahan koneksi aktif ketika fungsi externalCall dijalankan. Bahaya penahanan soket ini cakupannya mengikuti test starvation lab, membuktikan bahwa penundaan defer conn.Close() selama I/O eksternal secara langsung memblokir antrean peminjam lain.

Holding open connections across unbounded network boundaries creates artificial throughput bottlenecks.

internal/pool/service.go:36-57

Verifikasi Test: Membuktikan Penalti Pembuatan Koneksi Langsung

Membandingkan latensi eksekusi antara SetMaxIdleConns(0) dan pool aktif.

```
func TestDirectConnectionOverhead(t *testing.T) {
    mockDriver := pool.NewMockDriver(100, 5*time.Millisecond)
    unpooledDB := pool.OpenDB(mockDriver)
    unpooledDB.SetMaxIdleConns(0) // Memaksa pembukaan koneksi baru setiap kali
    // Eksekusi 5 query unpooled menghasilkan akumulasi delay 25ms+
    pooledDB := pool.OpenDB(mockDriver)
    pooledDB.SetMaxIdleConns(5)
    pooledDB.SetMaxOpenConns(5)
    // Eksekusi 5 query pooled memakai ulang koneksi idle yang sama
    if pooledDuration >= unpooledDuration { t.Errorf("expected pooled to be faster") }
}
```

TestDirectConnectionOverhead membuktikan bahwa pool dengan idle connections selalu lebih cepat daripada pembukaan koneksi baru di tiap query. Menjalankan query pada database tidak otomatis menghasilkan performa optimal tanpa konfigurasi idle pool yang memadai.

Reusing warm connection handles bypasses socket setup latencies, ensuring microsecond query dispatch.

tests/pool_test.go:13-49

Verifikasi Test: Pool Terlalu Besar Menghabiskan Kapasitas Server

Membuktikan penolakan deterministik ketika pool klien melebihi kapasitas backend.

```
func TestOversizedPoolExhaustsServerConnections(t *testing.T) {
    mockDriver := pool.NewMockDriver(10, 0) // Server hanya izinkan 10 slot
    db := pool.OpenDB(mockDriver)
    db.SetMaxOpenConns(20) // Klien salah konfigurasi dengan pool 20
    // 20 goroutine meminta koneksi secara bersamaan
    wg.Wait()
    if errCount < 10 {
        t.Errorf("expected at least 10 connection attempts to fail, got %d", errCount)
    }
}
```

Pengujian ini membuktikan bahwa pool klien berukuran 20 pada server berkapasitas 10 selalu menghasilkan penolakan deterministik. Hanya pada jalur ini kegagalan server terbukti terjadi karena aplikasi mengabaikan batas kapasitas fisik backend database.

Oversized client pools inevitably trigger server-side connection exhaustion rejections under load.

tests/pool_test.go:51–86

Verifikasi Test: Kebocoran Satu Worker Melumpuhkan Worker Lain

Permintaan kedua timeout karena koneksi tunggal tersandera I/O eksternal.

```
func TestConnectionStarvationDueToLeak(t *testing.T) {
    db.SetMaxOpenConns(1) // Pool ketat berukuran 1 koneksi
    go func() {
        _ = svc.ProcessOrderUnsafeLeak(ctx, 1, func() error {
            close(acquired); time.Sleep(100 * time.Millisecond); return nil
        })
    }()
    <-acquired
    ctx, cancel := context.WithTimeout(ctx, 20*time.Millisecond)
    defer cancel()
    err := svc.ProcessOrderSafe(ctx, 2, nil) // Gagal timeout 20ms
}
```

Unit test mereproduksi pool starvation secara deterministik pada pool berukuran satu koneksi. Terjadinya timeout pada pemanggil yang aman bukan berarti kodenya cacat, melainkan akibat soket tunggal tersandera oleh fungsi lain yang ceroboh menjalankan delay jaringan eksternal.

Unreleased connection handles monopolize shared pools, causing collateral starvation for valid requests.

Verifikasi Test: Bahaya Deadlock Akuisisi Koneksi Bersarang

Meminta koneksi kedua pada thread yang sama saat pool berukuran 1 memicu hang.

```
func TestPoolLockingDeadlock(t *testing.T) {
    db.SetMaxOpenConns(1) // Pool berukuran 1
    ctx, cancel := context.WithTimeout(context.Background(), 50*time.Millisecond)
    defer cancel()
    conn1, err := db.Conn(ctx)
    if err != nil { t.Fatalf("failed to acquire first: %v", err) }
    defer conn1.Close()
    _, err = db.Conn(ctx) // Meminta koneksi kedua pada pool yang sama!
    if err == nil { t.Fatalf("expected context deadline exceeded, got nil") }
}
```

Pengujian ini memvalidasi skenario pool-locking di mana sebuah thread mencoba mengakuisisi koneksi kedua sementara koneksi pertama belum dilepas. Verifikasi ini cakupannya mengikuti test lab, memvalidasi formula Tn x (Cm - 1) + 1 sebagai syarat matematis mutlak pencegah kebuntuan pool.

Nested connection acquisition without sufficient pool headroom results in deterministic thread self-deadlock.

tests/pool_test.go:154–174

Checklist Manajemen Koneksi di Lingkungan Produksi

Protokol wajib arsitektur sebelum merilis pool database ke lingkungan berskala tinggi.

- ✓ Hitung total armada: (instance_count x pool_size) wajib lebih kecil dari max_connections.
- ✓ Sisihkan 15 slot koneksi untuk maintenance, superuser, dan physical replication.
- ✓ Pastikan tidak ada panggilan I/O jaringan atau komputasi file di dalam blok koneksi.
- ✓ Gunakan defer conn.Close() atau helper sql.DB tingkat tinggi pada setiap percabangan.
- ✓ Pasang context timeout ketat pada setiap peminjaman soket dan eksekusi query SQL.
- ✓ Aktifkan leak detection threshold (HikariCP / wrapper) untuk menangkap stack trace leak.

Enforce strict operational standards across connection sizing, lifecycle scoping, and timeout boundaries.

Anti-Pattern Umum dalam Arsitektur Connection Pool

Kekeliruan konfigurasi yang kerap melumpuhkan stabilitas database relasional.

POLA BERBAHAYA YANG HARUS DIHINDARI

Naikkan Pool Saat Error

Merespons error koneksi dengan memperbesar pool; memperparah CPU thrashing dan lock contention.

POLA BERBAHAYA YANG HARUS DIHINDARI

Sizing Per-Instance

Menghitung pool secara lokal tanpa mengalikan jumlah replica pod pada orkestrasi Kubernetes.

POLA BERBAHAYA YANG HARUS DIHINDARI

Bukan Berarti Gratis

Menganggap koneksi idle gratis bukan berarti tanpa beban; RAM backend tetap terkuras permanen.

Menaikkan ukuran pool saat database mengalami galat koneksi adalah anti-pattern paling umum yang memperparah insiden produksi. Mengabaikan konsumsi memori koneksi idle bukan berarti sistem bebas beban, melainkan mengunci ratusan megabyte RAM backend yang semestinya dipakai untuk buffer cache.

Avoid reactive pool enlargement and account for cumulative connection costs across container replicas.

Reproduksi Mandiri di Mesin Lokal Menggunakan Go

Verifikasi mandiri seluruh perilaku pool tanpa dependensi database eksternal.

```
cd labs/19-database-connection-pooling
go test -v ./...    # menjalankan 10 test suite verifikasi overhead, limit, dan leak
go test -race ./... # pembuktian sinkronisasi konkurensi bersih tanpa data race
go run ./cmd/demo   # eksekusi demonstrasi interaktif skenario 1, 2, dan 3
```

Seluruh arsitektur dibangun murni menggunakan pustaka standar Go tanpa perlu menginstal server PostgreSQL lokal. Pengujian ini memvalidasi mekanika peminjaman soket, namun production masih perlu mengonfigurasi parameter PgBouncer serta pemantauan metrik pg_stat_activity.

Self-contained standard library harnesses enable rapid, zero-dependency validation of connection architectures.

README.md:1-35

Prinsip Inti: Pool Lebih Kecil, Sistem Lebih Cepat

Stabilitas sistem basis data bertumpu pada disiplin pembatasan konkurensi.

TIGA PILAR KETAHANAN KONEKSI SISTEM

Batas Sizing Konservatif

Jadikan kapasitas hardware sebagai batas atas; batasi antrean di aplikasi alih-alih di database.

TIGA PILAR KETAHANAN KONEKSI SISTEM

Isolasi I/O Ketat

Keluarkan seluruh panggilan eksternal dari penahanan koneksi; pinjam hanya saat query siap.

TIGA PILAR KETAHANAN KONEKSI SISTEM

Skalabilitas Horizontal

Tidak otomatis menaikkan slot; gunakan proxy pooling transaksi untuk ribuan klien.

Pertanyaan mendasar dalam rekayasa perangkat lunak bukanlah seberapa besar pool yang bisa dibuat, melainkan sekecil apa pool yang masih mampu memenuhi target throughput. Menghadapi lonjakan traffic tidak otomatis diselesaikan dengan menaikkan koneksi server, melainkan dengan memasang proxy pooling transaksi.

Resilient architectures maintain compact connection pools, isolate external I/O, and scale via proxies.

Koneksi database adalah sumber daya terbatas backend, bukan variabel yang bisa diperbesar tanpa batas.

Database connections are finite physical resources that demand strict lifecycle governance and proxy mediation.

Software Engineering Lab 19 – Database Connection Pooling

internal/pool/mockdb.go • internal/pool/service.go
cmd/demo/main.go • tests/pool_test.go
engineering/01–design.md • content/02–master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/19-database-connection-pooling`

Lukman / lukman–ss

Commit: b2d3274 • Status: APPROVED
Test passing: 10/10 test suites (Overhead, Oversized, Starvation, Deadlock, Reuse)
Race detector: Clean (go test -race ./... PASS in 1.328s)
Zero external dependencies: 100% Go standard library.